

Dockerhub push

2. Jenkins Credentials Setup

Go to **Jenkins** → **Manage Jenkins** → **Credentials** → **Global** → **Add Credential**

Add these two credentials:

Docker Hub credentials:

Field	Value
Kind	Username with password
ID	<code>dockerhub-credentials</code>
Username	Your Docker Hub username
Password	Your Docker Hub Access Token (not password)

Get Docker Hub token at: `hub.docker.com` → **Account Settings** → **Security** → **New Access Token**

3. GitHub Variables to Add/Update

Go to **GitHub** → **Settings** → **Environments** → **dev**, add:

Variable	Value
<code>DOCKERHUB_USERNAME</code>	Your Docker Hub username
<code>IMAGE_TAG</code>	<code>latest</code> (or <code>\${{ github.sha }}</code> for unique tags)

Updating jenkins pipeline script

4. docker-compose.yml — pull from Docker Hub instead of building

Update your `docker-compose.yml` to use the pre-built images:

```
pipeline {
```

agent any

options {

 disableConcurrentBuilds()

}

parameters {

 string(name: 'REMOTE_HOST', defaultValue: '40.81.224.228', description: 'Remote Azure VM IP - client test vm')

 string(name: 'REMOTE_USER', defaultValue: 'azurevps1', description: 'remote ssh username of client test vm')

 string(name: 'APP_DIR', defaultValue: '/home/azurevps1/event-mgt-system-api', description: 'App Directory on Remote client test VM')

 string(name: 'REPO_URL', defaultValue: 'git@github.com:chamalkaMarasinghe/event-management-system-api.git', description: 'Git Repository URL')

 string(name: 'BRANCH', defaultValue: 'main', description: 'Git Branch to deploy')

 string(name: 'DOCKERHUB_USERNAME', defaultValue: 'chamalkamarasinghe', description: 'Docker Hub account user name')

 string(name: 'DOPPLER_PROJECT', defaultValue: 'event-mgt-api', description: 'Doppler Project Name')

 string(name: 'DOPPLER_CONFIG_GATEWAY_SERVICE', defaultValue: 'dev_api_gateway', description: 'Doppler Config (env)')

 string(name: 'DOPPLER_CONFIG_AUTH_SERVICE', defaultValue: 'dev_auth_service', description: 'Doppler Config (env)')

 string(name: 'DOPPLER_CONFIG_EVENT_SERVICE', defaultValue: 'dev_event_service', description: 'Doppler Config (env)')

 string(name: 'DOPPLER_CONFIG_NOTIFICATION_SERVICE', defaultValue: 'dev_notification_service', description: 'Doppler Config (env)')

}

environment {

 SSH_OPTIONS = "-o StrictHostKeyChecking=no"

```
}
```

```
stages {
```

```
  stage('Validate SSH Connection') {
```

```
    steps {
```

```
      sshagent(credentials: ['azure-ssh-key']) {
```

```
        sh "ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} 'echo  SSH connection successful'"
```

```
      }
```

```
    }
```

```
  }
```

```
  stage('Clone or Pull Code on Remote VM') {
```

```
    steps {
```

```
      sshagent(credentials: ['azure-ssh-key']) {
```

```
        sh """
```

```
          ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '
```

```
          set -e
```

```
          if [ ! -d "${params.APP_DIR}/.git" ]; then
```

```
            echo "  Cloning repository from branch: ${params.BRANCH}"
```

```
            rm -rf ${params.APP_DIR}
```

```
            git clone --branch ${params.BRANCH} ${params.REPO_URL} ${params.APP_DIR}
```

```
          else
```

```
            echo "  Pulling latest code from branch: ${params.BRANCH}"
```

```
            cd ${params.APP_DIR}
```

```
            git fetch origin
```

```
git checkout ${params.BRANCH}
```

```
git reset --hard origin/${params.BRANCH}
```

```
fi
```

```
,
```

```
""
```

```
}
```

```
}
```

```
}
```

```
stage('Fetch .env from Doppler') {
```

```
steps {
```

```
withCredentials([string(credentialsId: 'doppler-token', variable: 'DOPPLER_TOKEN')]) {
```

```
sshagent(credentials: ['azure-ssh-key']) {
```

```
sh ""
```

```
ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '
```

```
set -e
```

```
export DOPPLER_TOKEN=${DOPPLER_TOKEN}
```

```
echo "🔑 Fetching environment variables from Doppler..."
```

```
cd ${params.APP_DIR}
```

```
doppler secrets download --project ${params.DOPPLER_PROJECT} --config
```

```
${params.DOPPLER_CONFIG_GATEWAY_SERVICE} --format env --no-file > services/api-gateway/.env
```

```
doppler secrets download --project ${params.DOPPLER_PROJECT} --config
```

```
${params.DOPPLER_CONFIG_AUTH_SERVICE} --format env --no-file > services/auth-service/.env
```

```
doppler secrets download --project ${params.DOPPLER_PROJECT} --config
```

```
${params.DOPPLER_CONFIG_EVENT_SERVICE} --format env --no-file > services/event-service/.env
```

```
doppler secrets download --project ${params.DOPPLER_PROJECT} --config
```

```
${params.DOPPLER_CONFIG_NOTIFICATION_SERVICE} --format env --no-file > services/notification-service/.env
```

```
echo "✅ .env file created"
```

```
,
```

```
""
```

```
}
```

```
}
```

```
}
```

```
}
```

```
stage('Docker Compose Build & Deploy') {
```

```
  steps {
```

```
    sshagent(credentials: ['azure-ssh-key']) {
```

```
      sh """
```

```
ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '
```

```
set -e
```

```
echo "🔧 Running docker-compose with memory limit..."
```

```
cd ${params.APP_DIR}
```

```
export DOCKER_BUILDKIT=1
```

```
echo "🔧 Building API Gateway image..."
```

```
docker build -t ${params.DOCKERHUB_USERNAME}/event-mgt-api-gateway:latest -f services/api-gateway/Dockerfile services/api-gateway
```

```
echo "🔧 Building Auth Service image..."
```

```
docker build -t ${params.DOCKERHUB_USERNAME}/event-mgt-auth-service:latest -f services/auth-service/Dockerfile services/auth-service
```

```
echo "🔨 Building Event Service image..."
```

```
docker build -t ${params.DOCKERHUB_USERNAME}/event-mgt-event-service:latest -f
services/event-service/Dockerfile services/event-service
```

```
echo "🔨 Building Notification Service image..."
```

```
docker build -t ${params.DOCKERHUB_USERNAME}/event-mgt-notification-service:latest -f
services/notification-service/Dockerfile services/notification-service
```

```
echo "✅ All images built successfully"
```

```
,
```

```
""
```

```
}
```

```
}
```

```
}
```

```
stage('Push Images to Docker Hub') {
```

```
  steps {
```

```
    withCredentials([usernamePassword(
```

```
      credentialsId: 'dockerhub-credentials',
```

```
      usernameVariable: 'DOCKERHUB_USER',
```

```
      passwordVariable: 'DOCKERHUB_PASS'
```

```
    ])) {
```

```
      sshagent(credentials: ['azure-ssh-key']) {
```

```
        sh """
```

```
          ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '
```

```
          set -e
```

```
echo "🔑 Logging in to Docker Hub..."
```

```
echo "${DOCKERHUB_PASS}" | docker login -u "${DOCKERHUB_USER}" --password-stdin
```

```
echo "📦 Pushing API Gateway image..."
```

```
docker push ${params.DOCKERHUB_USERNAME}/event-mgt-api-gateway:latest
```

```
echo "📦 Pushing Auth Service image..."
```

```
docker push ${params.DOCKERHUB_USERNAME}/event-mgt-auth-service:latest
```

```
echo "📦 Pushing Event Service image..."
```

```
docker push ${params.DOCKERHUB_USERNAME}/event-mgt-event-service:latest
```

```
echo "📦 Pushing Notification Service image..."
```

```
docker push ${params.DOCKERHUB_USERNAME}/event-mgt-notification-service:latest
```

```
echo "🔑 Logging out from Docker Hub..."
```

```
docker logout
```

```
echo "✅ All images pushed to Docker Hub"
```

```
,
```

```
""
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
post {  
    always {  
        cleanWs()  
    }  
}  
}
```

5. Pipeline Flow `` Push to main ↓ GitHub Actions → triggers Jenkins ↓ **Jenkins**: 1.  Validate SSH to azurevps1 2.  Clone/Pull code on azurevps1 3.  Fetch .env files from Doppler 4.  Build Docker images on azurevps1 5.  Push images to Docker Hub 6.  docker compose pull + up -d

Finalized pipeline -----

```
pipeline {  
    agent any  
  
    options {  
        disableConcurrentBuilds()  
    }  
  
    parameters {  
        string(name: 'REMOTE_HOST', defaultValue: '40.81.224.228', description: 'Remote Azure VM IP - client  
test vm')  
  
        string(name: 'REMOTE_USER', defaultValue: 'azurevps1', description: 'remote ssh username of client test  
vm')  
  
        string(name: 'APP_DIR', defaultValue: '/home/azurevps1/event-mgt-system-api', description: 'App Directory  
on Remote client test VM')  
  
        string(name: 'REPO_URL', defaultValue: 'git@github.com:chamalkaMarasinghe/event-management-system-  
api.git', description: 'Git Repository URL')  
  
        string(name: 'BRANCH', defaultValue: 'main', description: 'Git Branch to deploy')  
  
        string(name: 'DOCKERHUB_USERNAME', defaultValue: 'chamalkamarasinghe', description: 'Docker Hub  
account user name')  
  
        string(name: 'DOPPLER_PROJECT', defaultValue: 'event-mgt-api', description: 'Doppler Project Name')  
  
        string(name: 'DOPPLER_CONFIG_GATEWAY_SERVICE', defaultValue: 'dev_api_gateway', description:  
'Doppler Config (env)')  
  
        string(name: 'DOPPLER_CONFIG_AUTH_SERVICE', defaultValue: 'dev_auth_service', description:  
'Doppler Config (env)')  
  
        string(name: 'DOPPLER_CONFIG_EVENT_SERVICE', defaultValue: 'dev_event_service', description:  
'Doppler Config (env)')  
  
        string(name: 'DOPPLER_CONFIG_NOTIFICATION_SERVICE', defaultValue: 'dev_notification_service',  
description: 'Doppler Config (env)')  
    }  
}
```

```
environment {  
  
    SSH_OPTIONS = "-o StrictHostKeyChecking=no"  
  
}
```

```
stages {  
  
    stage('Validate SSH Connection') {  
  
        steps {  
  
            sshagent(credentials: ['azure-ssh-key']) {  
  
                sh "ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} 'echo '  
SSH connection successful"  
  
            }  
  
        }  
  
    }  
  
}
```

```
stage('Clone or Pull Code on Remote VM') {  
  
    steps {  
  
        sshagent(credentials: ['azure-ssh-key']) {  
  
            sh ""  
  
            ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '  
  
            set -e  
  
            if [ ! -d "${params.APP_DIR}/.git" ]; then  
  
                echo "  Cloning repository from branch: ${params.BRANCH}"  
  
                rm -rf ${params.APP_DIR}  
  
                git clone --branch ${params.BRANCH} ${params.REPO_URL} ${params.APP_DIR}  
  
            else
```

```
echo "🔄 Pulling latest code from branch: ${params.BRANCH}"
```

```
cd ${params.APP_DIR}
```

```
git fetch origin
```

```
git checkout ${params.BRANCH}
```

```
git reset --hard origin/${params.BRANCH}
```

```
fi
```

```
,
```

```
"""
```

```
}
```

```
}
```

```
}
```

```
stage('Fetch .env from Doppler') {
```

```
  steps {
```

```
    withCredentials([string(credentialsId: 'doppler-token', variable: 'DOPPLER_TOKEN')]) {
```

```
      sshagent(credentials: ['azure-ssh-key']) {
```

```
        sh """
```

```
          ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '
```

```
          set -e
```

```
          export DOPPLER_TOKEN=${DOPPLER_TOKEN}
```

```
          echo "🔑 Fetching environment variables from Doppler..."
```

```
          cd ${params.APP_DIR}
```

```
          doppler secrets download --project ${params.DOPPLER_PROJECT} --config
```

```
${params.DOPPLER_CONFIG_GATEWAY_SERVICE} --format env --no-file > services/api-gateway/.env
```

```
          doppler secrets download --project ${params.DOPPLER_PROJECT} --config
```

```
${params.DOPPLER_CONFIG_AUTH_SERVICE} --format env --no-file > services/auth-service/.env
```

```
    doppler secrets download --project ${params.DOPPLER_PROJECT} --config  
    ${params.DOPPLER_CONFIG_EVENT_SERVICE} --format env --no-file > services/event-service/.env
```

```
    doppler secrets download --project ${params.DOPPLER_PROJECT} --config  
    ${params.DOPPLER_CONFIG_NOTIFICATION_SERVICE} --format env --no-file > services/notification-  
service/.env
```

```
    echo "✅ .env file created"
```

```
    '
```

```
    ""
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
stage('Docker Compose Build & Deploy') {
```

```
  steps {
```

```
    sshagent(credentials: ['azure-ssh-key']) {
```

```
      sh ""
```

```
      ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '
```

```
      set -e
```

```
      echo "🔧 Running docker-compose with memory limit..."
```

```
      cd ${params.APP_DIR}
```

```
      export DOCKER_BUILDKIT=1
```

```
      echo "🔧 Building API Gateway image..."
```

```
      docker build -t ${params.DOCKERHUB_USERNAME}/event-mgt-api-gateway:latest -f services/api-  
gateway/Dockerfile services/api-gateway
```

```
echo "🔨 Building Auth Service image..."
```

```
docker build -t ${params.DOCKERHUB_USERNAME}/event-mgt-auth-service:latest -f  
services/auth-service/Dockerfile services/auth-service
```

```
echo "🔨 Building Event Service image..."
```

```
docker build -t ${params.DOCKERHUB_USERNAME}/event-mgt-event-service:latest -f  
services/event-service/Dockerfile services/event-service
```

```
echo "🔨 Building Notification Service image..."
```

```
docker build -t ${params.DOCKERHUB_USERNAME}/event-mgt-notification-service:latest -f  
services/notification-service/Dockerfile services/notification-service
```

```
echo "✅ All images built successfully"
```

```
,
```

```
""""
```

```
}
```

```
}
```

```
}
```

```
stage('Push Images to Docker Hub') {
```

```
  steps {
```

```
    withCredentials([usernamePassword(
```

```
      credentialsId: 'dockerhub-credentials',
```

```
      usernameVariable: 'DOCKERHUB_USER',
```

```
      passwordVariable: 'DOCKERHUB_PASS'
```

```
    )) {
```

```
      sshagent(credentials: ['azure-ssh-key']) {
```

```
sh ""
```

```
ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '
```

```
set -e
```

```
echo "🔑 Logging in to Docker Hub..."
```

```
echo "${DOCKERHUB_PASS}" | docker login -u "${DOCKERHUB_USER}" --password-stdin
```

```
echo "🚀 Pushing API Gateway image..."
```

```
docker push ${params.DOCKERHUB_USERNAME}/event-mgt-api-gateway:latest
```

```
echo "🚀 Pushing Auth Service image..."
```

```
docker push ${params.DOCKERHUB_USERNAME}/event-mgt-auth-service:latest
```

```
echo "🚀 Pushing Event Service image..."
```

```
docker push ${params.DOCKERHUB_USERNAME}/event-mgt-event-service:latest
```

```
echo "🚀 Pushing Notification Service image..."
```

```
docker push ${params.DOCKERHUB_USERNAME}/event-mgt-notification-service:latest
```

```
echo "🔑 Logging out from Docker Hub..."
```

```
docker logout
```

```
echo "✅ All images pushed to Docker Hub"
```

```
,
```

```
""
```

```
}
```

```
}  
  
}  
  
}
```

```
stage('Deploy with Docker Compose') {  
  steps {  
    sshagent(credentials: ['azure-ssh-key']) {  
      sh """  
        ssh ${env.SSH_OPTIONS} ${params.REMOTE_USER}@${params.REMOTE_HOST} '  
        set -e  
        echo " 🚀 Deploying application..."  
        cd ${params.APP_DIR}  
  
        # Pull latest images from Docker Hub  
        docker-compose pull || docker compose pull  
  
        # Restart containers with new images  
        docker-compose down --remove-orphans  
        docker-compose up -d || docker compose up -d  
  
        echo " ✅ Application deployed successfully"  
  
        # Cleanup unused Docker resources  
        echo " 🧹 Cleaning up unused Docker resources..."  
        docker system prune -f
```

,

""

}

}

}

}

post {

always {

cleanWs()

}

}

}

Docker compose-----

```
version: "3.9"
```

```
services:
```

```
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    restart: always
    ports:
      - "5672:5672"
      - "15672:15672"
    networks:
      - microservices-net
```

```
  api-gateway:
    # build: ./services/api-gateway
    image: chamalakamarasinghe/event-mgt-api-gateway:latest
    container_name: api-gateway
    restart: always
    ports:
      - "5004:5004"
    env_file:
      - ./services/api-gateway/.env
    depends_on:
      - auth-service
      - events-service
    networks:
      - microservices-net
```

```
  auth-service:
    # build: ./service/auth-service
    image: chamalakamarasinghe/event-mgt-auth-service:latest
    container_name: auth-service
    restart: always
    env_file:
      - ./services/auth-service/.env
    ports:
      - "5005:5005"
    depends_on:
      - rabbitmq
    networks:
      - microservices-net
```

```
events-service:
  # build: ./services/event-service
  image: chamalakamarasinghe/event-mgt-event-service:latest
  container_name: event-service
  restart: always
  env_file:
    - ./services/event-service/.env
  ports:
    - "5006:5006"
  depends_on:
    - rabbitmq
  networks:
    - microservices-net

notification-service:
  # build: ./services/notification-service
  image: chamalakamarasinghe/event-mgt-notification-service:latest
  container_name: notification-service
  restart: always
  env_file:
    - ./services/notification-service/.env
  ports:
    - "5007:5007"
  depends_on:
    - rabbitmq
  networks:
    - microservices-net

networks:
  microservices-net:
    driver: bridge
```